

Feed Forward Neural Networks

S. R. Mahadeva Prasanna

Professor, Dept of DSAI
Indian Institute of Information Technology Dharwad

prasanna@iiitdwd.ac.in



August 23, 2024

Some Good Resources for Deep Learning

- Duda and Hart, "Pattern Classification" Wiley, 2001.
- C. M. Bishop, "Neural Networks for Pattern Recognition", 1995.
- B. Yegnanarayana, "Artificial Neural Networks", PHI, 1999.
- C. M. Bishop, "PRML", Springer, 2006.
- Michael Nielsen "Neural Networks and Deep Learning" Open Book.
- Ian Goodfellow, et. al. , "Deep Learning", MIT Press, 2016.
- Charu Agarwal, "NNDL", Springer 2018.
- Mithesh M. Khapra, "Deep Learning Part I and II" NPTEL Course.
- Ali Ghodsi, "Deep Learning", University of Waterloo Course.
- C. M. Bishop & H. Bishop, "Deep Learning", Springer, 2024

What is FFNN?

- **Layers** of processing units, each layer **feeding** input to the next layer in a **feedforward manner** through a set of weights.
- With suitable choice of architecture and neurons, it is possible to perform several pattern recognition tasks.
- **Two layer** FFNN with **linear** units.
- **Two layer** FFNN with **nonlinear** units in the output layer.
- **Multilayer** FFNN with **non-linear** processing units.
- **Deep** FFNN with **non-linear** processing units.



Two Layer FFNN with Nonlinear Units

- A set or group of input patterns assigned an output pattern, then output patterns can be viewed as distinct classes or class labels.
- There are no restrictions on no of input patterns.
- Input-output pattern vector pairs $(\mathbf{a}_l, \mathbf{b}_l)$, $l = 1, 2, \dots, L$ in this case can be considered as a training set for a pattern classification task.
- **A two layer FFNN with nonlinear (hard-limiting) output function for units in output layer can be used to perform task of pattern classification.**
- The neuron with hard-limiting output fn is termed as perceptron.
- Number of output units depends on the number of distinct classes.



Two Layer FFNN with Nonlinear Units

- For a given set of input-output patterns, weighted sum of input values is compared with threshold for the unit to determine whether sum is greater or less than threshold.
- A set of inequalities are generated with the given data.
- No unique solution for the weights in this case.
- Necessary to determine a set of weights to satisfy all the inequalities.
- Determination of such weights is accompanied by means of incremental adjustment of weights using a learning law.



Pattern Classification Problem

- If a subset of input patterns belong to one class (say class A_1) and the remaining subset to another class (say class A_2), then determine a set of weights $w_1, w_2, \dots, w_M$ such that if the weighted sum
- $\sum_{i=1}^M w_i a_i > \theta$, then $\mathbf{a}$ belongs to class A_1
- $\sum_{i=1}^M w_i a_i \leq \theta$, then $\mathbf{a}$ belongs to class A_2
- Note that dividing surface between two classes is given by $\sum_{i=1}^M w_i a_i = \theta$
- This eqn represents a linear hyperplane in M dimensional space.
- Hyperplane is a point if $M = 1$, a str. line if $M = 2$, and a plane if $M = 3$.



Pattern Classification Problem

- Since solution of classification problem involves determining weights and threshold value, input a_0 to the connection involving the threshold value $w_0 = \theta$ is always -1.
- Defining the augmented input and weight vectors as $\mathbf{a} = (-1, a_1, \dots, a_M)^T$ and $\mathbf{w} = (w_0, w_1, \dots, w_M)^T$, respectively, the perceptron classification problem can be stated as follows:
- If $\mathbf{w}^T \mathbf{a} > 0$, then $\mathbf{a}$ belongs to class A_1 , and if $\mathbf{w}^T \mathbf{a} \leq 0$, then $\mathbf{a}$ belongs to class A_2 .
- The equation for the dividing linear hyperplane is $\mathbf{w}^T \mathbf{a} = 0$.



Perceptron Learning Law

- Input space M dimensional and number of output patterns are two, corresponding to the two classes.
- We use $(M + 1)$ dimensional vector to denote a point in the M -dimensional space, as the a_0 component of the vector is always -1 .
- Let subsets A_1 and A_2 of points in M dimensional space contain sample patterns belonging to classes A_1 and A_2 , respectively.
- Objective in perceptron learning is to systematically adjust weights for each presentation of an input vector belonging to A_1 or A_2 , along with its class identification.



Perceptron Learning Law

- Perceptron learning law for two-class problem may be stated as follows:
- $\mathbf{w}(\mathbf{m} + 1) = \mathbf{w}(\mathbf{m}) + \eta \mathbf{a}$, if $a \in A_1$ and $\mathbf{w}^T(\mathbf{m})\mathbf{a} \leq 0$
- $\mathbf{w}(\mathbf{m} + 1) = \mathbf{w}(\mathbf{m}) - \eta \mathbf{a}$, if $a \in A_2$ and $\mathbf{w}^T(\mathbf{m})\mathbf{a} > 0$
- where index m is used to denote learning process at the m th step.
- Vectors $\mathbf{a}$ and $\mathbf{w}(\mathbf{m})$ are input and weight vectors, respectively, at m th step, and η is a positive learning rate parameter.
- Note that NO adjustment of weights is made when the input vector is correctly classified. That is,
- $\mathbf{w}(\mathbf{m} + 1) = \mathbf{w}(\mathbf{m})$, if $a \in A_1$ and $\mathbf{w}^T(\mathbf{m})\mathbf{a} > 0$
- $\mathbf{w}(\mathbf{m} + 1) = \mathbf{w}(\mathbf{m})$, if $a \in A_2$ and $\mathbf{w}^T(\mathbf{m})\mathbf{a} \leq 0$

Perceptron Learning Law

- Equation of straight line as a decision boundary separating the two classes is given by $w_1 a_1 + w_2 a_2 = \theta$
- For different values of weights during learning, position of line changes.
- The two classes can be separated by a straight line, and hence they are called linearly separable classes.
- **If the examples are not linearly separable, the straight line wanders in the plane during learning, and weights do not converge to a final stable value, as the two classes cannot be separated by a single straight line.**
- **PS** : Perceptron convergence theorem - 'perceptron learning law converges to a final set of weight values in a finite number of steps, if the classes are linearly separable'.

Gradient Descent Method for Minimization

- More generic method for minimization
- Given function $G(\theta_0, \theta_1)$
- Start with some (θ_0, θ_1)
- Iteratively change (θ_0, θ_1) to minimize $G(\theta_0, \theta_1)$ until minima is reached.
- Values of (θ_0, θ_1) at minima are chosen as optimal parameters.



Gradient Descent Algorithm

- $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} G(\theta_0, \theta_1), j = 0, 1$
- α is a learning rate parameter.
- Partial derivative essentially represents slope
- $\theta_0 := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} G(\theta_0, \theta_1)$
- $\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} G(\theta_0, \theta_1)$
- Repeat the above two steps until convergence
- Smaller α , smaller step size
- Bigger α , bigger step size
- Bigger step size may lead to non-convergence to minima.
- Also large α in some case may diverge.



Perceptron Learning as Gradient Descent

- Change in weights proportional to negative derivative of an error surface.
- The perceptron learning law can also be written as $w(m+1) = w(m) + \eta(b(m) - s(m))a(m)$.
- $b(m)$ is the desired output, which for the binary case is given by, $b(m) = 1$, if $a(m) \in A_1$ and $b(m) = 0$, if $a(m) \in A_2$
- $s(m)$ is actual output for input vector $a(m)$ to perceptron.
- Actual output is given by, $s(m) = 1$, if $w^T(m)a(m) > 0$ and $s(m) = 0$, if $w^T(m)a(m) \leq 0$
- Note that if $s(m) = b(m)$, then $w(m+1) = w(m)$, i.e., no correction takes place.
- If there is an error, $s(m) \neq b(m)$, then the weight update happens.

Perceptron Learning as Gradient Descent

- $w(m+1) = w(m) + \eta(b(m) - s(m))a(m)$.
- $w(m+1) = w(m) + \eta e(m)a(m)$, where $e(m) = (b(m) - s(m))$ is error.
- Define performance measure as
 $E(m) = -e(m)x(m) = -e(m)w^T(m)a(m)$, $x(m)$ is activation function & $E(m)$ becomes squared error function.
- The negative derivative of $E(m)$ w.r.t. the weight vector $w(m)$ is
 $-\frac{\partial E(m)}{\partial W(m)} = e(m)a(m)$
- Thus weight update $\eta e(m)a(m)$ in perceptron learning is proportional to negative gradient of performance measure $E(m)$.



Perceptron Representation Problem

- Convergence only if the classes are linearly separable.
- For linearly separable classes, perceptron convergence theorem gives final set of weights reached in a finite number of steps.
- These weights define a linear hyperplane separating the two classes.
- For $M = 2$ dim, binary i/p vector, no. of fns possible, $2^{2^M} = 16$
- Ex, OR, AND, NOR, NAND, ... are linearly separable functions.
- XOR and XNOR are not linearly separable.
- No. of linearly separable fns. decrease as i/p dim is increased.
- These linearly **inseparable** problems do not lead to convergence of weights through the perceptron learning law
- These problems are not representable by a single layer of perceptron.

Linear Inseparability: Hard Problems

- A two-layer FFNN with hard-limiting threshold units in the output layer can solve linearly separable pattern classification problems.
- This is also called a single layer perceptron, as there is only one layer of nonlinear units.
- There are many problems which are not linearly separable, and hence are not representable by a single layer perceptron.
- These unrepresentable problems are called **hard problems**.
- If o/p of second layer combined forming another layer, then any convex region can be formed by the separating surfaces.
- A convex region is one in which a line joining any two points is entirely confined to the region itself.



Linear Separability: Hard Problems

- No. of units in second layer determines shape of dividing surface. Number of units in third layer decides number of classes.
- Three-layer network is not general enough, as it is not guaranteed that class regions in pattern space form convex regions in all cases.
- Ex, desired classes are enclosed by complicated nonconvex regions.
- Note that intersection of linear hyperplanes in the three layer network can only produce convex surfaces.
- However, intersection of the convex regions may produce any nonconvex region also.
- Adding one more layer of units to combine outputs of third layer can yield surfaces which can separate even nonconvex regions



Linear Separability: Hard Problems

- A four-layer network with the input layer consisting of linear units, and the other three layers consisting of hard-limiting nonlinear units, can perform any complex pattern classification tasks.
- Thus all the hard problems mentioned earlier can be handled by a multilayer feedforward neural network, with nonlinear units. Such a network is also called a **multilayer perceptron**.
- Similar behaviour is expected from a multilayer feedforward neural network when the output functions of the units are continuous nonlinear functions, such as sigmoid functions.
- **PS:** With illustrations understand how adding more layers of perceptrons helps dealing with hard problems.



Hard Learning Problem

- Pattern mapping involves finding weights, given a set of input-output pattern pairs.
- It is necessary to know desired output for each unit in the hidden and output layers.
- Once desired output is known, error, from each unit may be used to guide updating of weights leading to unit from units in previous layer.
- Desired output only for units in the output layer, and not for units in hidden layers.
- A learning rule that depends on difference between desired and actual outputs is not feasible in this case.
- Problem of updating weights is called a hard learning problem.



Hard Learning Problem: Differentiable Output Function

- Hard learning problem is solved by using a differentiable nonlinear output function for each unit in the hidden and output layers.
- The corresponding learning law is based on propagating the error from the output layer to the hidden layers for updating the weights.
- This is an error correcting learning law, also called the generalized delta rule.
- It is based on the principle of gradient descent along the error surface.
- **PS:** Understand difference between hard problem and hard learning problem



Backpropagation Learning

- Learning alg for multilayer FFNN.
- Also termed as generalized delta rule.
- A gradient descent along error surface to arrive at optimum set of weights.
- Error is the squared difference between desired output and actual output obtained at the output layer of network.
- Estimate of gradient descent along error surface to increment weight connecting the units j and i is given by:
- $\Delta w_{ij} = -\eta \frac{\partial E(m)}{\partial w_{ij}}$, where $\eta > 0$ is a learning rate parameter.
- $w_{ij}(m+1) = w_{ij} + \Delta w_{ij}$



Why Generalized Delta Rule ?

- Deriving expressions for Δw_{ij} for different layers.
- Consider a MLFFNN with 3 layers, input, one hidden and output.
- First layer has I linear input units indexed by i , second layer has J nonlinear units indexed by j , and third layer has K nonlinear units indexed by k .
- $a(m)$ is given at the input layer and the desired output $b(m)$ is available only at the output layer.
- Error between desired output vector $b(m)$ and the actual output vector $b'(m)$ is available only at the output layer.
- **Use this error to adjust weights from input units to hidden units (w_{ji}^h), and also weights (w_{kj}) from hidden units to output units.**

Backpropagation Learning Algorithm

- The mean squared error at the m th step is given by
- $E(m) = \frac{1}{2} \sum_{k=1}^K (b_k(m) - b'_k(m))^2 = \frac{1}{2} \sum_{k=1}^K (b_k(m) - s_k^o)^2 = \frac{1}{2} \sum_{k=1}^K (b_k(m) - f_k^o(x_k^o))^2$
- $x_k^o = \sum_{j=1}^J w_{kj} s_j^h$; $s_j^h = f_j^h(x_j^h)$; $x_j^h = \sum_{i=1}^I w_{ji} s_i$; $s_i = x_i = a_i(m)$
- For the weights leading to the units in the output layer:
$$\Delta w_{kj} = -\eta \frac{\partial E(m)}{\partial w_{kj}}$$
- $\frac{\partial E(m)}{\partial w_{kj}} = \frac{1}{2} \sum_{k=1}^K \frac{\partial}{\partial w_{kj}} [b_k(m) - f_k^o(\sum_{j=1}^J w_{kj} s_j^h)]^2 = -\sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o s_j^h = -\sum_{k=1}^K \delta_k^o s_j^h$, where $\delta_k^o = [b_k(m) - f_k^o] \dot{f}_k^o$
- $\Delta w_{kj} = \eta \delta_k^o s_j^h$
- $w_{kj}(m+1) = w_{kj}(m) + \eta \delta_k^o s_j^h$



Backpropagation Learning Algorithm

- For the weights leading to the units in the hidden layer:

$$\Delta w_{ji}^h = -\eta \frac{\partial E(m)}{\partial w_{ji}^h}$$

- $$\frac{\partial E(m)}{\partial w_{ji}^h} = \frac{1}{2} \sum_{k=1}^K \frac{\partial}{\partial w_{ji}^h} [b_k(m) - f_k^o(\sum_{j=1}^J w_{kj} s_j^h)]^2 =$$
$$- \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kj} \frac{\partial s_j^h}{\partial w_{ji}^h}$$

- $$s_j^h = f_j^h(x_j^h) \implies \frac{\partial s_j^h}{\partial w_{ji}^h} = \dot{f}_j^h \frac{\partial x_j^h}{\partial w_{ji}^h}$$

- $$x_j^h = \sum_{i=1}^I w_{ji}^h s_i \implies \frac{\partial x_j^h}{\partial w_{ji}^h} = s_i$$

- $$\frac{\partial E(m)}{\partial w_{ji}^h} = - \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kj} \dot{f}_j^h s_i = -\delta_j^h s_i,$$

Backpropagation Learning Algorithm

- $\delta_j^h = \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kj} \dot{f}_j^h = \dot{f}_j^h \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kj} = \dot{f}_j^h \sum_{k=1}^K w_{kj} \delta_k^o$
- $\delta_j^h = \dot{f}_j^h \sum_{k=1}^K w_{kj} \delta_k^o$
- $\Delta w_{ji}^h = \eta \delta_j^h s_i = \eta \delta_j^h a_i \implies \Delta w_{ji}^h(m) = \eta \delta_j^h a_i(m)$
- $w_{ji}(m+1) = w_{ji}(m) + \Delta w_{ji}^h(m) = w_{ji}(m) + \eta \delta_j^h a_i(m)$
- $\delta_j^h = \dot{f}_j^h \sum_{k=1}^K w_{kj} \delta_k^o$ represents error propagated back to output of hidden units from next layer.
- **Hence the name backpropagation for this learning algorithm.**
- **PS:** Work out math for BP algorithm with 2 hidden layers.



Backpropagation Learning Algorithm for 2 Hidden Layers

- The mean squared error at the m th step is given by
- $E(m) = \frac{1}{2} \sum_{k=1}^K (b_k(m) - s_k^o)^2$
- $s_k^o = f_k^o(\sum_{l=1}^L w_{kl} s_l)$
- For the weights leading to the units in the output layer:
$$\Delta w_{kl} = -\eta \frac{\partial E(m)}{\partial w_{kl}}$$
- $$\frac{\partial E(m)}{\partial w_{kl}} = \frac{1}{2} \sum_{k=1}^K \frac{\partial}{\partial w_{kl}} [b_k(m) - f_k^o(\sum_{l=1}^L w_{kl} s_l)]^2 =$$

$$-\sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o s_l = -\sum_{k=1}^K \delta_k^o s_l, \text{ where}$$

$$\delta_k^o = [b_k(m) - f_k^o] \dot{f}_k^o$$
- $\Delta w_{kl} = \eta \delta_k^o s_l$
- $w_{kl}(m+1) = w_{kl}(m) + \eta \delta_k^o s_l$

Backpropagation Learning Algorithm for 2 Hidden Layers

- For the weights leading to the units in the hidden layer 2:

$$\Delta w_{lj}^h = -\eta \frac{\partial E(m)}{\partial w_{lj}^h}$$

- $$\frac{\partial E(m)}{\partial w_{lj}^h} = \frac{1}{2} \sum_{k=1}^K \frac{\partial}{\partial w_{lj}^h} [b_k(m) - f_k^o(\sum_{l=1}^L w_{kl} s_l^h)]^2 =$$
$$- \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} \frac{\partial s_l^h}{\partial w_{lj}^h}$$

- $$s_l^h = f_l^h(x_l^h) \implies \frac{\partial s_l^h}{\partial w_{lj}^h} = \dot{f}_l^h \frac{\partial x_l^h}{\partial w_{lj}^h}$$

- $$x_l^h = \sum_{j=1}^J w_{lj}^h s_j \implies \frac{\partial x_l^h}{\partial w_{lj}^h} = s_j$$

- $$\frac{\partial E(m)}{\partial w_{lj}^h} = - \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} \dot{f}_l^h s_j = -\delta_l^h s_j,$$



Backpropagation Learning Algorithm for 2 Hidden Layers

- $\delta_l^h = \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} \dot{f}_l^h = \dot{f}_l^h \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} = \dot{f}_l^h \sum_{k=1}^K w_{kl} \delta_k^o$
- $\delta_l^h = \dot{f}_l^h \sum_{k=1}^K w_{kl} \delta_k^o$
- $\Delta w_{lj}^h = \eta \delta_l^h s_j = \eta \delta_l^h s_j$
- $w_{ji}(m+1) = w_{ji}(m) + \Delta w_{ji}^h(m)$
- $\delta_l^h = \dot{f}_l^h \sum_{k=1}^K w_{kl} \delta_k^o$ represents error propagated back to output of hidden units from next layer.
- **Hence the name backpropagation for this learning algorithm.**

Backpropagation Learning Algorithm for 2 Hidden Layers

- For the weights leading to the units in the hidden layer 1:

$$\Delta w_{ji}^h = -\eta \frac{\partial E(m)}{\partial w_{ji}^h}$$

$$\begin{aligned} \frac{\partial E(m)}{\partial w_{ji}^h} &= \frac{1}{2} \sum_{k=1}^K \frac{\partial}{\partial w_{ji}^h} [b_k(m) - f_k^o(\sum_{l=1}^L w_{kl} s_l^h)]^2 = \\ &= - \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} \sum_{l=1}^L \frac{\partial s_l^h}{\partial w_{ji}^h} \end{aligned}$$

$$s_l^h = f_l^h(x_l^h) \implies \frac{\partial s_l^h}{\partial w_{ji}^h} = \dot{f}_l^h \frac{\partial x_l^h}{\partial w_{ji}^h}$$

$$x_l^h = \sum_{j=1}^J w_{lj}^h s_j \implies \frac{\partial x_l^h}{\partial w_{ji}^h} = w_{lj}^h \frac{\partial s_j}{\partial w_{ji}^h}$$

$$s_j^h = f_j^h(x_j^h) \implies \frac{\partial s_j^h}{\partial w_{ji}^h} = \dot{f}_j^h \frac{\partial x_j^h}{\partial w_{ji}^h}$$

$$x_j^h = \sum_{i=1}^I w_{ji}^h s_i \implies \frac{\partial x_j^h}{\partial w_{ji}^h} = w_{ji}^h s_i$$

Backpropagation Learning Algorithm for 2 Hidden Layers

- $\frac{\partial E(m)}{\partial w_{ji}^h} = - \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} \sum_{l=1}^L \dot{f}_l^h w_{lj} \dot{f}_j^h s_j = -\delta_j^h s_i,$
- $\delta_j^h = \dot{f}_j^h \sum_{l=1}^J w_{lj} \delta_l^h$
- $\Delta w_{ji}^h = \eta \delta_j^h s_i = \eta \delta_j^h s_i$
- $w_{ji}(m+1) = w_{ji}(m) + \Delta w_{ji}^h(m)$
- $\delta_j^h = \dot{f}_j^h \sum_{l=1}^J w_{lj} \delta_l^h$ represents error propagated back to output of hidden units from next hidden layer.
- This in turn has error back propagated from output layer
- **Hence the name backpropagation for this learning algorithm.**

-Backpropagation Learning Algorithm for 2 Hidden Layers

- The mean squared error at the m th step is given by
- $E(m) = \frac{1}{2} \sum_{k=1}^K (b_k(m) - s_k^o)^2$
- $s_k^o = f_k^o(\sum_{l=1}^L w_{kl} s_l)$
- For the weights leading to the units in the output layer:
$$\Delta w_{kl} = -\eta \frac{\partial E(m)}{\partial w_{kl}}$$
- $$\frac{\partial E(m)}{\partial w_{kl}} = \frac{\partial}{\partial w_{kl}} \sum_{k=1}^K \frac{1}{2} [b_k(m) - f_k^o(\sum_{l=1}^L w_{kl} s_l)]^2 =$$

$$-[b_k(m) - f_k^o] \dot{f}_k^o s_l = -\delta_k^o s_l, \text{ where } \delta_k^o = [b_k(m) - f_k^o] \dot{f}_k^o$$
- $\Delta w_{kl} = \eta \delta_k^o s_l$
- $w_{kl}(m+1) = w_{kl}(m) + \eta \delta_k^o s_l$



-Backpropagation Learning Algorithm for 2 Hidden Layers

- For the weights leading to the units in the hidden layer 2:

$$\Delta w_{lj}^{h2} = -\eta \frac{\partial E(m)}{\partial w_{lj}^{h2}}$$

$$\begin{aligned} \bullet \quad \frac{\partial E(m)}{\partial w_{lj}^{h2}} &= \frac{1}{2} \sum_{k=1}^K \frac{\partial}{\partial w_{lj}^{h2}} [b_k(m) - f_k^o(\sum_{l=1}^L w_{kl} s_l^{h2})]^2 = \\ &= - \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} \frac{\partial s_l^{h2}}{\partial w_{lj}^{h2}} \end{aligned}$$

$$\bullet \quad s_l^{h2} = f_l^{h2}(x_l^{h2}) \implies \frac{\partial s_l^{h2}}{\partial w_{lj}^{h2}} = \dot{f}_l^{h2} \frac{\partial x_l^{h2}}{\partial w_{lj}^{h2}}$$

$$\bullet \quad x_l^{h2} = \sum_{j=1}^J w_{lj}^{h2} s_j \implies \frac{\partial x_l^{h2}}{\partial w_{lj}^{h2}} = s_j$$

$$\bullet \quad \frac{\partial E(m)}{\partial w_{lj}^{h2}} = - \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} \dot{f}_l^{h2} s_j = -\delta_l^{h2} s_j,$$

-Backpropagation Learning Algorithm for 2 Hidden Layers

- $\delta_l^{h2} = \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} \dot{f}_l^{h2} = \dot{f}_l^{h2} \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o w_{kl} = \dot{f}_l^{h2} \sum_{k=1}^K w_{kl} \delta_k^o$
- $\delta_l^{h2} = \dot{f}_l^{h2} \sum_{k=1}^K w_{kl} \delta_k^o$
- $\Delta w_{lj}^{h2} = \eta \delta_l^{h2} s_j = \eta \delta_l^{h2} s_j$
- $w_{ji}(m+1) = w_{ji}(m) + \Delta w_{lj}^{h2}(m)$
- $\delta_l^{h2} = \dot{f}_l^{h2} \sum_{k=1}^K w_{kl} \delta_k^o$ represents error propagated back to output of hidden units from next layer.
- **Hence the name backpropagation for this learning algorithm.**

-Backpropagation Learning Algorithm for 2 Hidden Layers

- For the weights leading to the units in the hidden layer 1:

$$\Delta w_{ji}^{h1} = -\eta \frac{\partial E(m)}{\partial w_{ji}^{h1}}$$

$$\begin{aligned} \bullet \frac{\partial E(m)}{\partial w_{ji}^{h1}} &= \frac{1}{2} \sum_{k=1}^K \frac{\partial}{\partial w_{ji}^{h1}} [b_k(m) - f_k^o(\sum_{l=1}^L w_{kl} s_l^{h2})]^2 = \\ &= -\sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o \sum_{l=1}^L w_{kl} \frac{\partial s_l^{h2}}{\partial w_{ji}^{h1}} \end{aligned}$$

$$\bullet s_l^{h2} = f_l^{h2}(x_l^{h2}) \implies \frac{\partial s_l^{h2}}{\partial w_{ji}^{h1}} = \dot{f}_l^{h2} \frac{\partial x_l^{h2}}{\partial w_{ji}^{h1}}$$

$$\bullet x_l^{h2} = \sum_{j=1}^J w_{lj}^{h2} s_j \implies \frac{\partial x_l^{h2}}{\partial w_{ji}^{h1}} = w_{lj}^{h2} \frac{\partial s_j}{\partial w_{ji}^{h1}}$$

$$\bullet s_j^{h1} = f_j^{h1}(x_j^{h1}) \implies \frac{\partial s_j^{h1}}{\partial w_{ji}^{h1}} = \dot{f}_j^{h1} \frac{\partial x_j^{h1}}{\partial w_{ji}^{h1}}$$

$$\bullet x_j^{h1} = \sum_{i=1}^I w_{ji}^{h1} s_i \implies \frac{\partial x_j^{h1}}{\partial w_{ji}^{h1}} = s_i$$

-Backpropagation Learning Algorithm for 2 Hidden Layers

- $\frac{\partial E(m)}{\partial w_{ji}^{h1}} = - \sum_{k=1}^K [b_k(m) - f_k^o] \dot{f}_k^o \sum_{l=1}^L w_{kl} \dot{f}_l^{h2} w_{lj}^{h2} \dot{f}_j^{h1} s_j = -\delta_j^{h1} s_i,$
- $\delta_j^{h1} = \dot{f}_j^{h1} \sum_{l=1}^L w_{lj}^{h2} \delta_l^{h2}$
- $\Delta w_{ji}^{h1} = \eta \delta_j^{h1} s_i$
- $w_{ji}(m+1) = w_{ji}(m) + \Delta w_{ji}^{h1}(m)$
- $\delta_j^{h1} = \dot{f}_j^{h1} \sum_{l=1}^L w_{lj}^{h2} \delta_l^{h2}$ represents error propagated back to output of hidden units from next hidden layer.
- This in turn has error back propagated from output layer
- **Hence the name backpropagation for this learning algorithm.**



Description and Features of Backpropagation

- Training patterns are applied in random order one by one.
- Each application of given training set of patterns is called a **cycle or epoch**.
- Involves propagation of error backwards in order to determine the update for weights leading to the units in a hidden layer.
- There is no feedback of the signal itself at any stage, as it is a feedforward neural network.
- Learning is exactly the same as the delta learning and hence it is also called generalized delta rule.
- The term generalized is used because the delta learning could be extended to the hidden layer units.



Description and Features of Backpropagation

- **Backpropagation of error is possible only if the output functions of the nonlinear processing units are differentiable.**
- If these output functions are linear, then we cannot realize the advantage of a multilayer network to generate complex decision boundaries for a nonlinearly separable (hard) classification problems.
- A common differentiable output function used in the backpropagation learning is one which possesses a **sigmoid nonlinearity**.
- Two examples of sigmoidal nonlinear function are the **logistic function and hyperbolic tangent function**.
- Logistic function: $f(x) = \frac{1}{1+e^{-x}}, -\infty < x < \infty \implies 0 \leq f(x) \leq 1$
- Hyperbolic function: $f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, -\infty < x < \infty \implies -1 \leq f(x) \leq 1$

Gradient Descent Nature of Backpropagation

- BP learning is based on gradient descent along the error surface.
- Weight adj. proportional to negative gradient of error wrt wt.
- Error is instant. error between desired and actual values of output.
- Instant. error assumed to be a sample function of a random process.
- Error can be assumed to be a random variable.
- Gradient descent method is a stochastic gradient learning method.



Gradient Descent Nature of Backpropagation

- Due to stoch. nature, path to min. of error surface will be zigzag.
- Error surface itself will be an approximation to true error surface determined by the entire training set of patterns.
- True error surface is not a smooth quadratic surface.
- Error surface may contain several local minima besides the global minimum.
- Stochastic approximation of gradient descent used in backpropagation learning need not converge.
- There is no proof of convergence even in the mean as in the case of the LMS algorithm.



Stopping Criteria for BP - Change in Gradient

- Since there is no proof of convergence, some heuristic criteria are used to stop the process of learning.
- Based on values of gradient and error in successive iterations and also on total number of iterations.
- Average gradient value over each training cycle or epoch is observed, and if this average value is below a preset threshold value for successive cycles, then the training process may be stopped.



Stopping Criteria for BP - Error Threshold

- Training process may be stopped using a threshold for the average error and observing the average error in successive cycles.
- Main objective is to capture implicit pattern behaviour in training set data so that adequate generalization takes place in the network.
- Generalization feature is verified by testing performance of network for several new (test) patterns.



Performance of Backpropagation Algorithm

- Performance depends on initial setting of weights, learning rate parameter, output functions of units, presentation of training data, besides specific pattern recognition task or application.
- Initial weights correspond to a priori knowledge.
- **Online mode** of adjustment is updating weights for each pattern pair.
- **Batch mode** is updating weights for all the pattern pairs.
- Too small η slow speed of learning and slow convergence.
- Too big η faster speed of learning, large random fluctuations, may lead to unstable condition and weights may not converge.
- **Overfitting** weights tuned more to the training data.



Limitations of Backpropagation Algorithm

- Slow convergence
- No guarantee of convergence
- Local minima

